

Split the Chain: Design and Deploy Your First LangChain Service

Component extracted: Document Retriever -> `RetrieverService.GetDocuments` (gRPC, Python + LangChain)

Requirement	Deliverable	File
1	gRPC Interface Contract	retriever_service/service.proto
2	Microservice implementation and test evidence	server.py, retriever.py, client_test.py, output below
3	Boundary reflection (150-250 words)	Section 3
4	Limitations and next steps	Section 4

Activity 1: Map the monolith

The starter script `techdocs_monolith.py` runs **query -> retriever -> LLM -> summarizer -> answer** in one process.

Component	Input	Output	Bound by
Retriever	query, top_k	list of documents	CPU / memory
LLM gateway	question + retrieved docs	raw answer text	I/O (network, rate limits)
Summarizer	raw answer text	short formatted answer	logic (cheap)

Why the retriever first

- Heavily reused: every Q&A; endpoint needs retrieval, so extracting it benefits all of them.
- Clean input/output boundary (query in, documents out), so the interface stays small.
- CPU-bound, unlike the I/O-bound LLM, so it needs to scale independently of it.
- Failures are easy to contain: the LLM service can still be called with cached context or a fallback.

1. gRPC Interface Contract (service.proto)

```
syntax = "proto3";

package techdocs.retriever.v1;

// First extracted microservice: the document retriever.
service RetrieverService {
    // Returns the top_k most relevant documents for a query.
    rpc GetDocuments (GetDocumentsRequest) returns (GetDocumentsResponse);
}

message GetDocumentsRequest {
    string query = 1;          // required, non-empty natural-language question
    int32 top_k = 2;          // optional, default 3, max 10
}

message Document {
    string id      = 1;        // stable document id
    string title   = 2;
    string content = 3;        // passage text passed on to the LLM service
    float score    = 4;        // relevance score in [0, 1]
}

message GetDocumentsResponse {
    repeated Document documents = 1;
    int32 latency_ms = 2;      // server-side time, useful for observability
}
```

Request and response schema, and how it reflects the service boundary

- **Request:** query (string, required) and top_k (int32, optional, default 3, max 10). Nothing about the LLM or summarizer leaks into the contract.
- **Response:** repeated Document with id, title, content, score, plus latency_ms for observability. Fields are typed; there are no raw JSON blobs.
- **Boundary:** the service owns *only* retrieval. The consumer decides what to do with the documents (prompting, summarising, caching), so each part can change or scale on its own.
- **Evolution:** new optional fields (e.g. filters) can be added with new field numbers without breaking existing callers.

2. Microservice Implementation and Test Evidence

server.py

```
"""gRPC RetrieverService. Stateless: no per-user state kept between requests."""
import logging, time
from concurrent import futures
import grpc
from grpc_reflection.v1alpha import reflection

import service_pb2, service_pb2_grpc
from retriever import retrieve

DEFAULT_TOP_K, MAX_TOP_K, PORT = 3, 10, 50051
log = logging.getLogger("retriever")

class RetrieverService(service_pb2_grpc.RetrieverServiceServicer):
    def GetDocuments(self, request, context):
        t0 = time.time()
        if not request.query.strip():
            context.abort(grpc.StatusCode.INVALID_ARGUMENT, "query must not be empty")
        top_k = request.top_k or DEFAULT_TOP_K
        if not 1 <= top_k <= MAX_TOP_K:
            context.abort(grpc.StatusCode.INVALID_ARGUMENT, f"top_k must be between 1 and {MAX_TOP_K}")
        try:
            docs = [service_pb2.Document(id=d.metadata["id"], title=d.metadata["title"], content=d.page_content, score=s)
                     for d, s in retrieve(request.query, top_k)]
        except Exception:
            log.exception("retrieval failed") # details in logs only
            context.abort(grpc.StatusCode.INTERNAL, "retrieval failed")
        ms = int((time.time() - t0) * 1000)
        log.info("GetDocuments query=%r top_k=%d returned=%d latency=%dms", request.query, top_k, len(docs), ms)
        return service_pb2.GetDocumentsResponse(documents=docs, latency_ms=ms)

def serve():
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=8))
    service_pb2_grpc.add_RetrieverServiceServicer_to_server(RetrieverService(), server)
    names = (service_pb2.DESCRIPTOR.services_by_name["RetrieverService"].full_name, reflection.SERVICE_NAME)
    reflection.enable_server_reflection(names, server) # lets grpcurl work without the .proto
    server.add_insecure_port(f"[::]:{PORT}")
    server.start()
    log.info("RetrieverService listening on localhost:%d", PORT)
    server.wait_for_termination()

if __name__ == "__main__":
    serve()
```

retriever.py (LangChain logic reused from the monolith)

```
"""LangChain retriever used by the service (keyword-overlap; swap for a vector store in production)."""
import sys, pathlib
sys.path.insert(0, str(pathlib.Path(__file__).resolve().parent.parent))
from techdocs_monolith import KeywordRetriever # reuse the exact monolith logic

def retrieve(query: str, top_k: int):
    q = set(query.lower().split())
    retriever = KeywordRetriever(k=top_k)
    for d in retriever.invoke(query):
        words = set(d.page_content.lower().split())
        score = len(q & words) / max(1, len(q))
        yield d, min(1.0, score)
```

Run and test commands

```
pip install grpcio grpcio-tools grpcio-reflection langchain-core
cd retriever_service
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. service.proto
python server.py
```

```
# grpcurl equivalent (reflection is enabled, so no .proto needed):
grpcurl -plaintext -d '{"query":"How do I reset my API key?","top_k":2}' \
  localhost:50051 techdocs.retriever.v1.RetrieverService/GetDocuments
```

Captured output (python client_test.py, run against the live server)

```
--- 1. valid request: query='How do I reset my API key?', top_k=2
documents {
  id: "doc-1"
  title: "Resetting API keys"
  content: "To reset your API key, open Settings > Security > API Keys and click Regenerate. Old keys stop working immediately."
  score: 0.285714298
}
documents {
  id: "doc-2"
  title: "Rate limits"
  content: "Rate limits are 100 requests per minute per key. Exceeding the limit returns HTTP 429 with a Retry-After header."
}
latency_ms: 10

--- 2. default top_k
['doc-3', 'doc-1', 'doc-2'] 0ms
--- 3. empty query -> ERROR INVALID_ARGUMENT: query must not be empty
--- 4. top_k=99 -> ERROR INVALID_ARGUMENT: top_k must be between 1 and 10
```

Server log

```
2026-10-02 05:20:31,297 INFO RetrieverService listening on localhost:50051
2026-10-02 05:20:32,821 INFO GetDocuments query='How do I reset my API key?' top_k=2 returned=2 latency=10ms
2026-10-02 05:20:32,826 INFO GetDocuments query='webhook retry backoff' top_k=3 returned=3 latency=0ms
```

How the test shows the service works

- **Responds without errors:** the valid call returns normally over a real gRPC channel.
- **Expected shape:** the response contains *Document* messages with id, title, content and score, plus latency_ms, matching the .proto.
- **LangChain logic runs:** the top hit for the API-key question is doc-1 (Resetting API keys), produced by the LangChain BaseRetriever; the server log records the call and its latency.
- **Failure behaviour:** an empty query and an out-of-range top_k return INVALID_ARGUMENT with a clear message instead of a crash.

3. Boundary Reflection (189 words)

I chose the retriever as the first microservice because it is the most reusable and the most self-contained part of the pipeline. Its contract is simple: a query and top_k go in, and a list of documents comes out. It is also CPU- and memory-bound, while the LLM call is I/O-bound, so the two have very different scaling needs. Today a slow OpenAI response freezes everything, including retrieval. Behind a gRPC boundary the retriever can run as its own deployment with its own replica count, tuned for CPU, and be redeployed without touching the rest of the system.

The typed .proto contract supports fault isolation. Callers set a deadline on every call, and the service answers bad input with explicit status codes (INVALID_ARGUMENT) instead of crashing, so a retriever failure surfaces as a clear error the orchestrator can retry, degrade or cache around. The service is stateless, so replicas can be added behind a load balancer. The trade-offs are an extra network hop on every query and some schema rigidity: new fields must be added carefully, using new field numbers and never reusing old ones, to stay backward compatible.

4. Limitations and Next Steps

Limitations and risks of the current design

- **Extra network hop:** every query now pays serialization and network latency; chatty call patterns would make this worse.
- **Prototype retriever:** keyword overlap over five in-memory documents is not semantic search and is not production quality.
- **Distributed-failure modes:** the orchestrator now has to handle timeouts, unavailable services and version mismatches that did not exist in-process.
- **Insecure transport and no auth:** the demo uses a plaintext local port.
- **Schema rigidity:** careless field changes (renumbering or reusing field numbers) would break callers.
- **Partial split:** the LLM and summarizer still live in the monolith, so its freezing problem is not solved yet.

Recommended next steps

- Replace the toy retriever with a real vector store (embeddings plus an index such as FAISS, pgvector or Qdrant) loaded once at startup.
- Add client-side deadlines, retries with backoff and a circuit breaker in the orchestrator, and fall back to cached results when the retriever is down.
- Add TLS/mTLS, authentication, the gRPC health-checking protocol, and metrics and tracing (latency, error rate, top_k distribution).
- Containerise and deploy behind a load balancer with autoscaling on CPU; version the API (package v1) and add contract tests.
- Extract the LLM gateway next (retries, rate limiting, token accounting, caching), then the summarizer, keeping each contract minimal and typed.